Intermediate representation

An **intermediate representation** (**IR**) is the data structure or code used internally by a compiler or virtual machine to represent source code. An IR is designed to be conducive to further processing, such as optimization and translation.[1] A "good" IR must be *accurate* – capable of representing the source code without loss of information[2] – and *independent* of any particular source or target language.[1] An IR may take one of several forms: an in-memory data structure, or a special tuple- or stack-based code readable by the program.[3] In the latter case it is also called an *intermediate language*.

A canonical example is found in most modern compilers. For example, the CPython interpreter transforms the linear human-readable text representing a program into an intermediate graph structure that allows flow analysis and re-arrangement before execution. Use of an intermediate representation such as this allows compiler systems like the GNU Compiler Collection and LLVM to be used by many different source languages to generate code for many different target architectures.

Intermediate language

An **intermediate language** is the language of an abstract machine designed to aid in the analysis of computer programs. The term comes from their use in compilers, where the source code of a program is translated into a form more suitable for code-improving transformations before being used to generate object or machine code for a target machine. The design of an intermediate language typically differs from that of a practical machine language in three fundamental ways:

- Each instruction represents exactly one fundamental operation; e.g. "shift-add" addressing modes common in microprocessors are not present.
- Control flow information may not be included in the instruction set.
- The number of processor registers available may be large, even limitless.

A popular format for intermediate languages is three-address code.

The term is also used to refer to languages used as intermediates by some high-level programming languages which do not output object or machine code themselves, but output the intermediate language only. This intermediate language is submitted to a compiler for such language, which then outputs finished object or machine code. This is usually done to ease the process of optimization or to increase portability by using an intermediate language that has compilers for many processors and operating systems, such as C. Languages used for this fall in complexity between high-level languages and low-level languages, such as assembly languages.

Languages

Though not explicitly designed as an intermediate language, C's nature as an abstraction of assembly and its ubiquity as the *de facto* system language in Unix-like and other operating systems has made it a popular intermediate language: Eiffel, Sather, Esterel, some dialects of Lisp (Lush, Gambit), Squeak's

Smalltalk-subset Slang, Nim, Cython, SystemTap, Vala, V, and others make use of C as an intermediate language. Variants of C have been designed to provide C's features as a portable assembly language, including C-- and the C Intermediate Language.

Any language targeting a virtual machine or p-code machine can be considered an intermediate language:

- Java bytecode
- Microsoft's Common Intermediate Language is an intermediate language designed to be shared by all compilers for the .NET Framework, before static or dynamic compilation to machine code.
- While most intermediate languages are designed to support statically typed languages, the Parrot intermediate representation is designed to support dynamically typed languages—initially Perl and Python.
- TIMI is used by compilers on the IBM i platform.
- O-code for BCPL
- MATLAB precompiled code
- Microsoft P-Code
- Pascal p-code

The GNU Compiler Collection (GCC) uses several intermediate languages internally to simplify portability and cross-compilation. Among these languages are

- the historical Register Transfer Language (RTL)
- the tree language GENERIC
- the SSA-based GIMPLE. (Lower-level than GENERIC; input for most optimizers; has a compact "bytecode" notation.)

GCC supports generating these IRs, as a final target:

- HSA Intermediate Layer
- LLVM Intermediate Representation (converted from GIMPLE in the now-defunct llvm-gcc which uses LLVM optimizers and codegen)

The LLVM compiler framework is based on the LLVM IR intermediate language, of which the compact, binary serialized representation is also referred to as "bitcode" and has been productized by Apple.[4][5] Like GIMPLE Bytecode, LLVM Bitcode is useful in link-time optimization. Like GCC, LLVM also targets some IRs meant for direct distribution, including Google's PNaCl IR and SPIR. A further development within LLVM is the use of *Multi-Level Intermediate Representation* (MLIR) with the potential to generate code for different heterogeneous targets, and to combine the outputs of different compilers.[6]

The ILOC intermediate language[7] is used in classes on compiler design as a simple target language.[8]

Other

Static analysis tools often use an intermediate representation. For instance, Radare2 is a toolbox for binary files analysis and reverse-engineering. It uses the intermediate languages ESIL[9] and REIL[10] to analyze binary files.

See also

- Abstract syntax tree – Tree representation of the abstract syntactic structure of source code
- BURS
- Bytecode – Instruction set designed to be run by a software interpreter
- Graph rewriting – Creating a new graph from an existing graph
- Interlingual machine translation – Type of machine translation
- Pivot language – Intermediary language between different languages
- Source-to-source compiler – Translator of computer source code
- Symbol table – Data structure used by a language translator such as a compiler or interpreter
- Term rewriting – Replacing subterm in a formula with another term
- UNCOL

References

1. Walker, David. "CS320: Compilers: Intermediate Representation" (http://www.cs.princeton.edu/courses/archive/spr03/cs320/notes/IR-trans1.pdf) (Lecture slides). Retrieved 12 February 2016.
2. Chow, Fred (22 November 2013). "The Challenge of Cross-language Interoperability" (https://queue.acm.org/detail.cfm?id=2544374). *ACM Queue*. **11** (10). Retrieved 12 February 2016.
3. Toal, Ray. "Intermediate Representations" (http://cs.lmu.edu/~ray/notes/ir/). Retrieved 12 February 2016.
4. "Bitcode (iOS, watchOS)" (https://news.ycombinator.com/item?id=9684223). Hacker News. 10 June 2015. Retrieved 17 June 2015.
5. "LLVM Bitcode File Format" (http://llvm.org/docs/BitCodeFormat.html). llvm.org. Retrieved 17 June 2015.
6. "MLIR" (https://mlir.llvm.org/).
7. "An ILOC Simulator" (http://www.engr.sjsu.edu/wbarrett/Parser/simManual.htm) Archived (https://web.archive.org/web/20090507084132/http://www.engr.sjsu.edu/wbarrett/Parser/simManual.htm) 2009-05-07 at the Wayback Machine by W. A. Barrett 2007, paraphrasing Keith Cooper and Linda Torczon, "Engineering a Compiler", Morgan Kaufmann, 2004. ISBN 1-55860-698-X.
8. "CISC 471 Compiler Design" (http://www.cis.udel.edu/~pollock/471/project2spec.pdf) by Uli Kremer
9. Radare2 Contributors. "ESIL" (https://web.archive.org/web/20150818235122/http://radare.gitbooks.io/radare2book/content/esil.html). Radare2 Project. Archived from the original (https://radare.gitbooks.io/radare2book/content/esil.html) on 18 August 2015. Retrieved 17 June 2015.
10. Sebastian Porst (7 March 2010). "The REIL language – Part I" (http://blog.zynamics.com/2010/03/07/the-reil-language-part-i/). zynamics.com. Retrieved 17 June 2015.

External links

- The Stanford SUIF Group